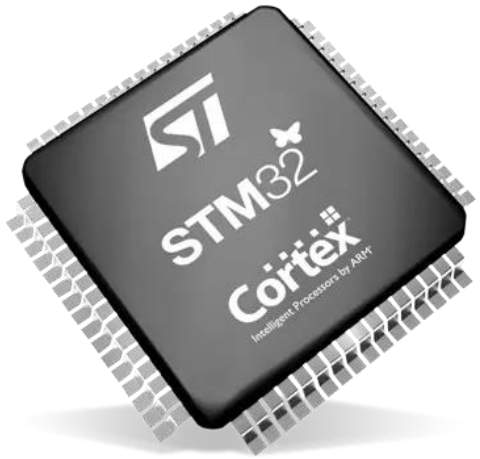


STM32 CubeIDE

Timers y UART STM32F4

Agenda

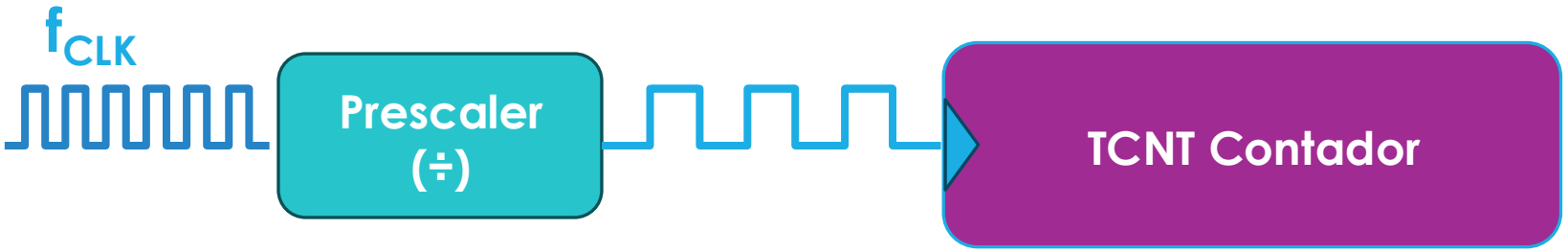
Timers

UART

Módulo Timers

Modo Temporizador

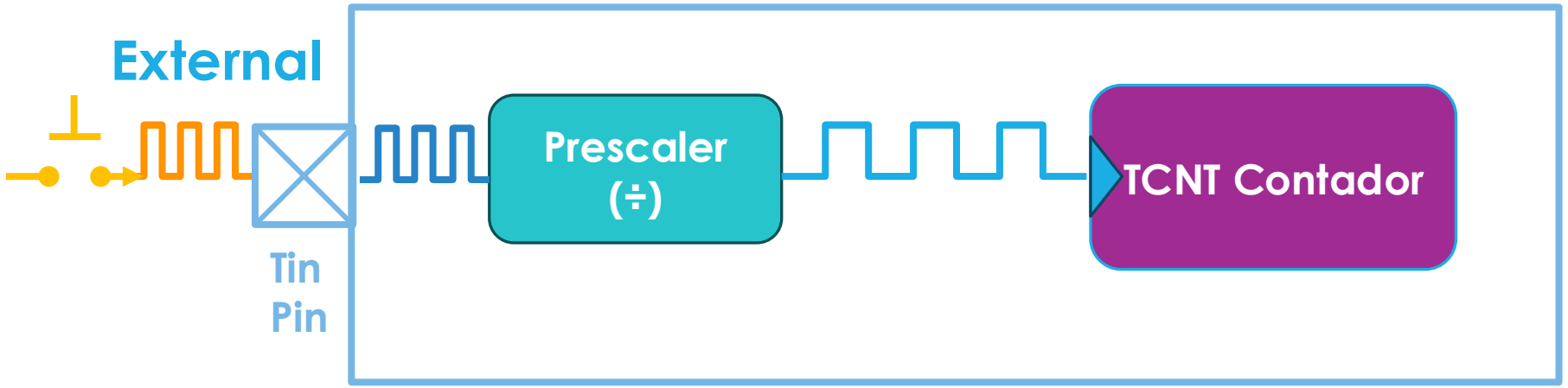
El timer no mide tiempo directamente, cuenta ciclos de reloj



$$f_{timer} = \frac{f_{APBx}}{(PSC + 1)} \qquad T_{overflow} = \frac{(PSC + 1)(ARR + 1)}{f_{timer}}$$

Modo Contador Eventos

Aquí el reloj interno se reemplaza por un reloj externo.



El pin Tin puede contar:

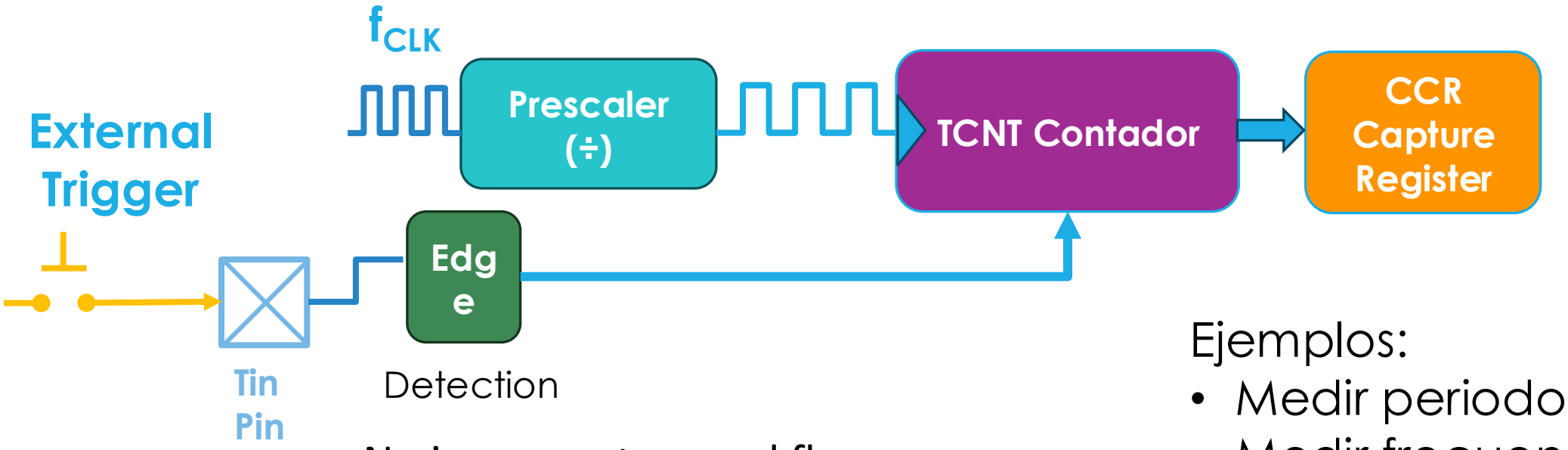
- flancos ascendentes
- descendentes
- ambos

Ejemplos:

- Conteo de pulsos de Encoder
- Conteo de piezas con infrarrojo

Modo Input Capture

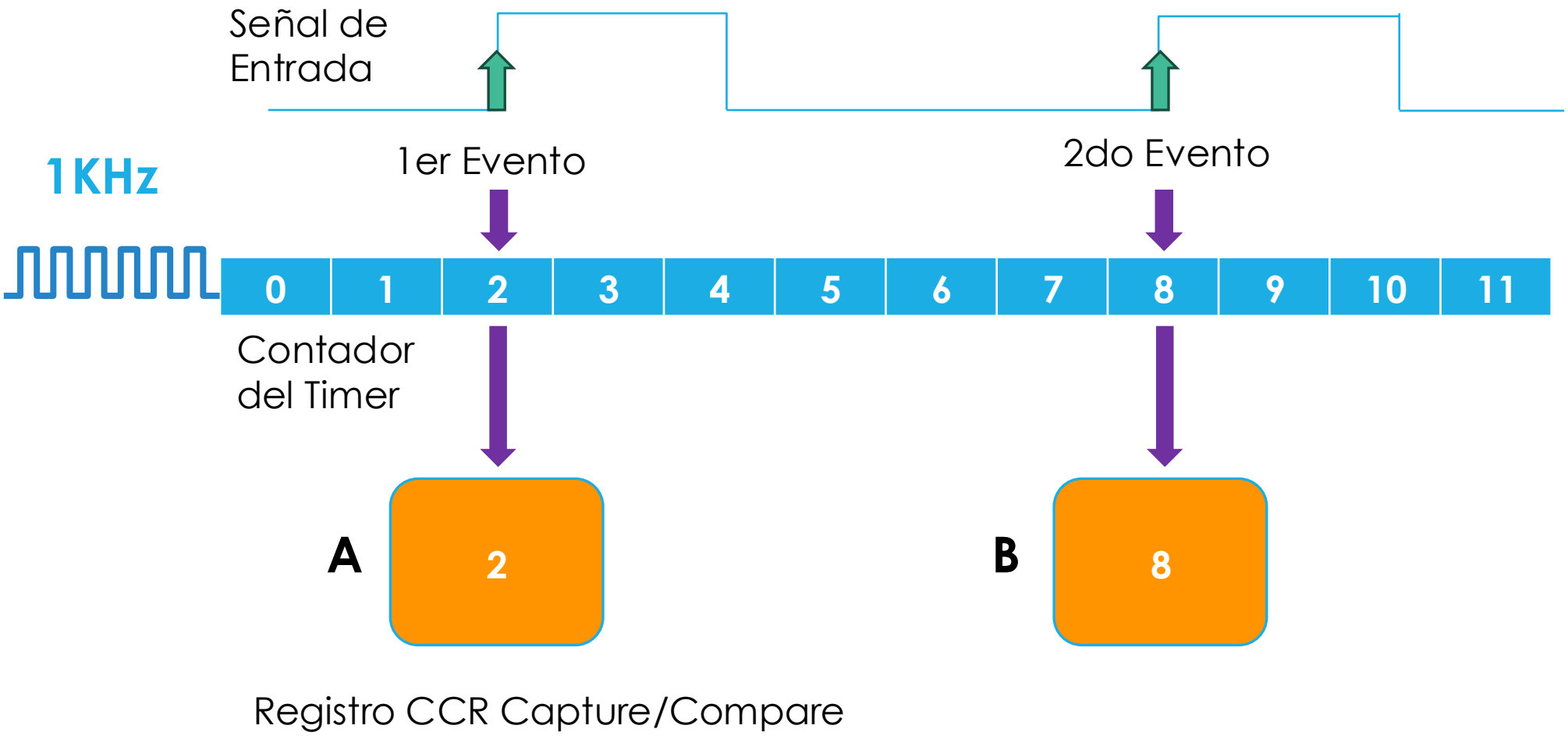
El timer sigue contando con su reloj interno. Cuando llega un pulso externo, guarda el valor actual del contador en un registro CCR.

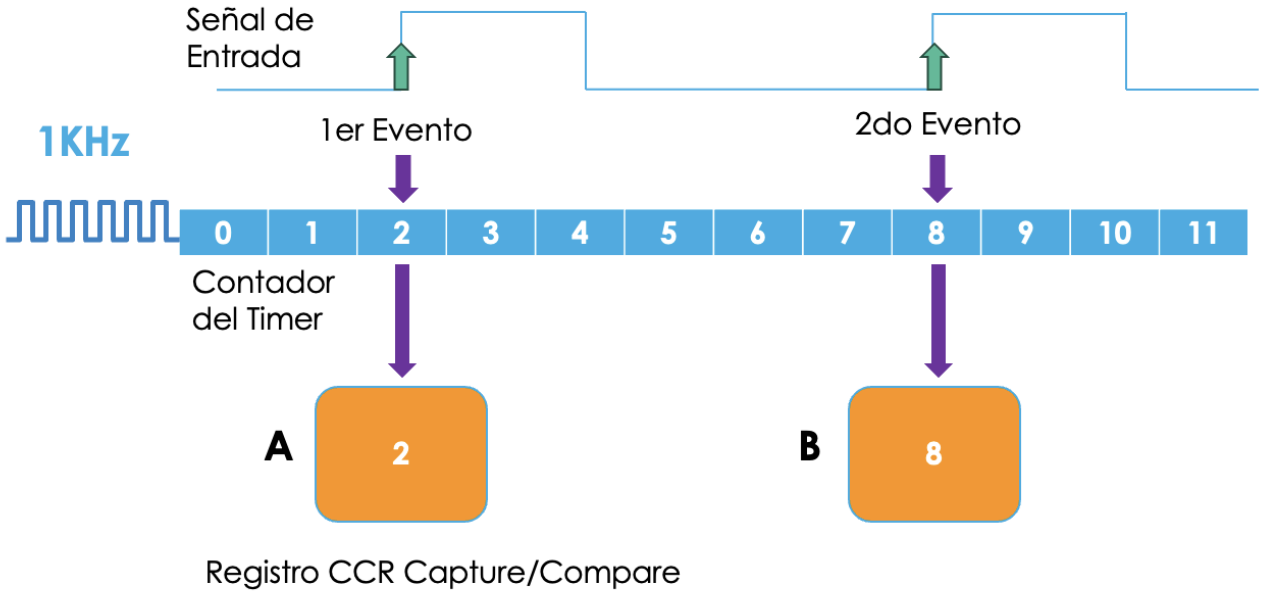


No incrementa por el flanco.
Solo “toma una foto” del contador.

Ejemplos:

- Medir periodo de una señal
- Medir frecuencia
- Medir ancho de pulso


Señal de
Entrada
1KHz
1er Evento
2do Evento
0
1
2
3
4
5
6
7
8
9
10
11
Contador
del Timer
A
2
B
8
Registro CCR Capture/Compare



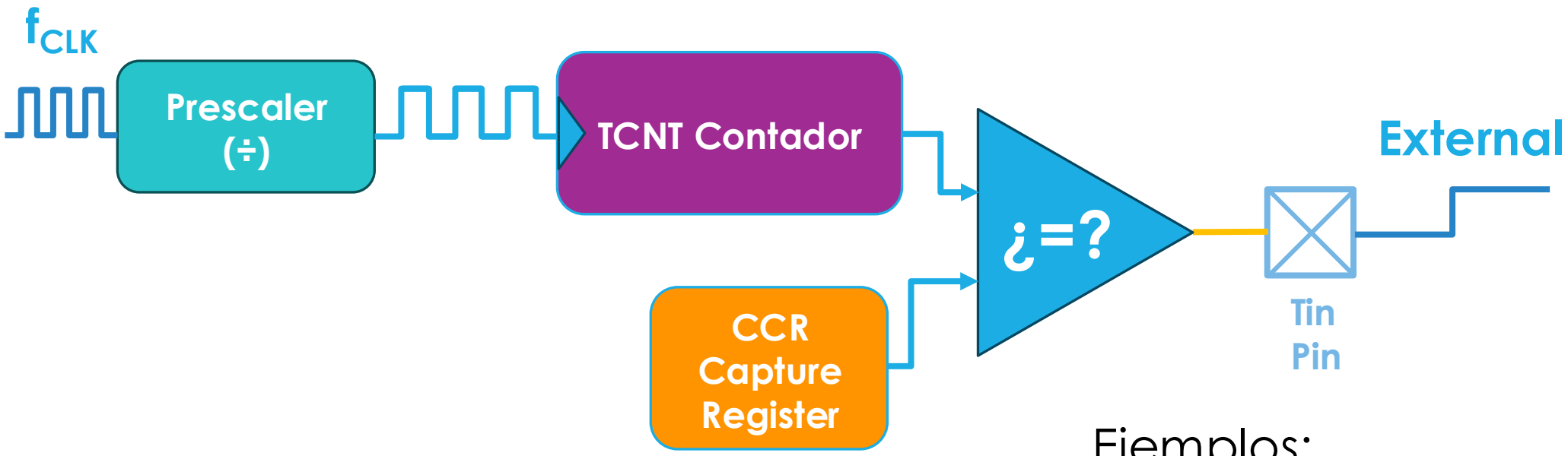
$$f_{TIM} = \frac{Timer_{clock}}{(TIM_{Prescaler} + 1)}; \quad T_{TIM} = \frac{1}{f_{TIM}}$$

$$T_{señal} = (CCR_B - CCR_A) * T_{TIM}$$

$$f_{señal} = \frac{f_{TIM}}{(CCR_B - CCR_A)}$$

Timers – Modo Output Compare

Compara continuamente el valor del contador (TCNTx) con el CCRx

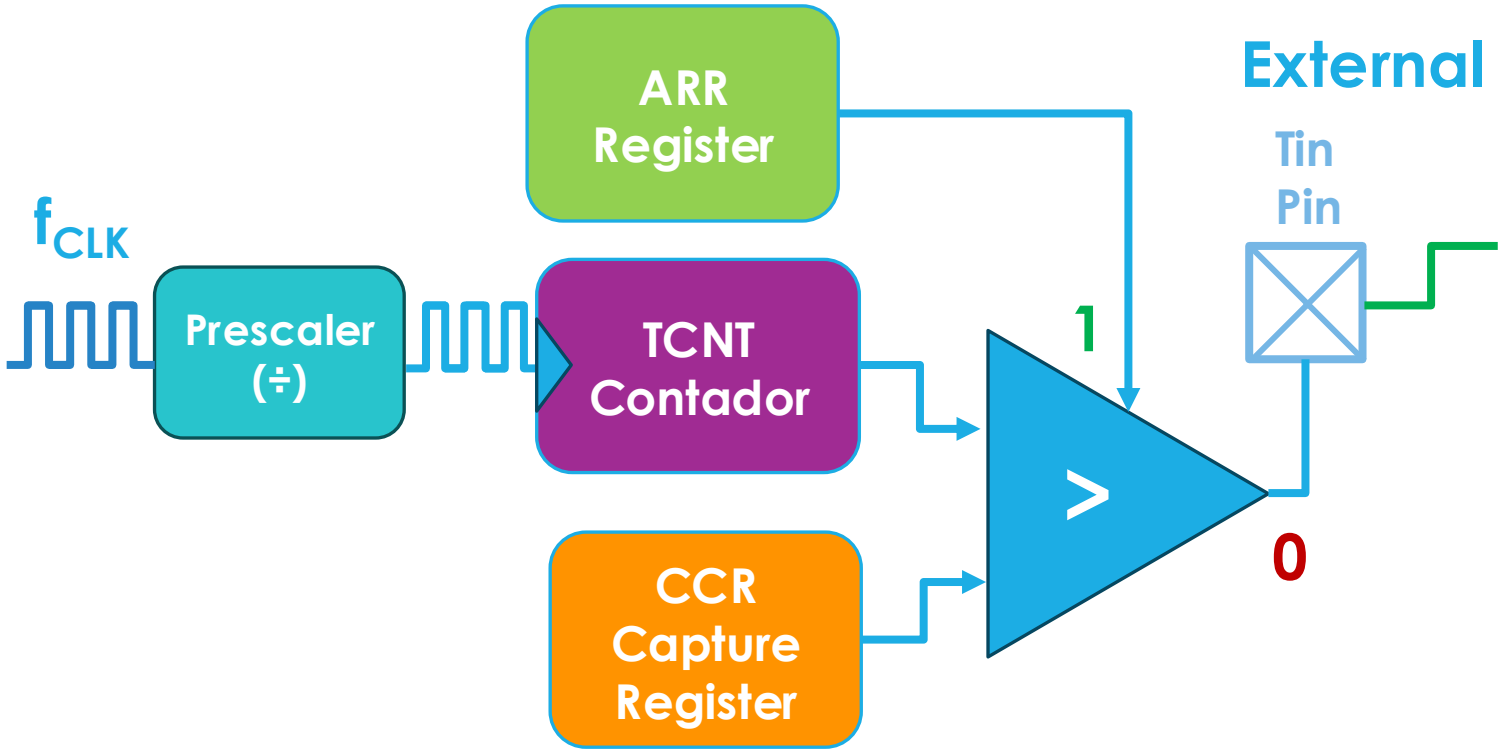


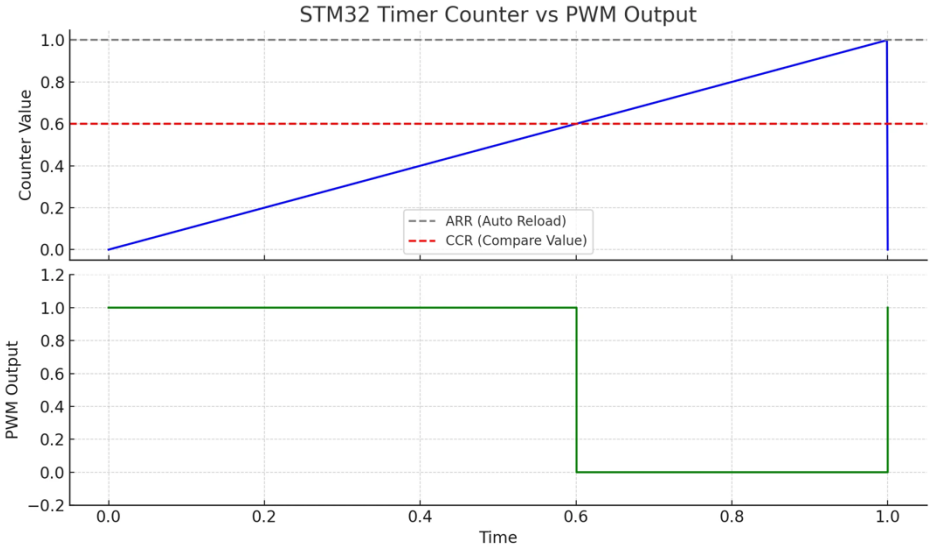
Ejemplos:

- Cambiar el estado de un pin
- Generar una interrupción
- Activar DMA
- Disparar otro periférico

Modo PWM

Compara continuamente el valor del contador (TCNTx) con el CCRx

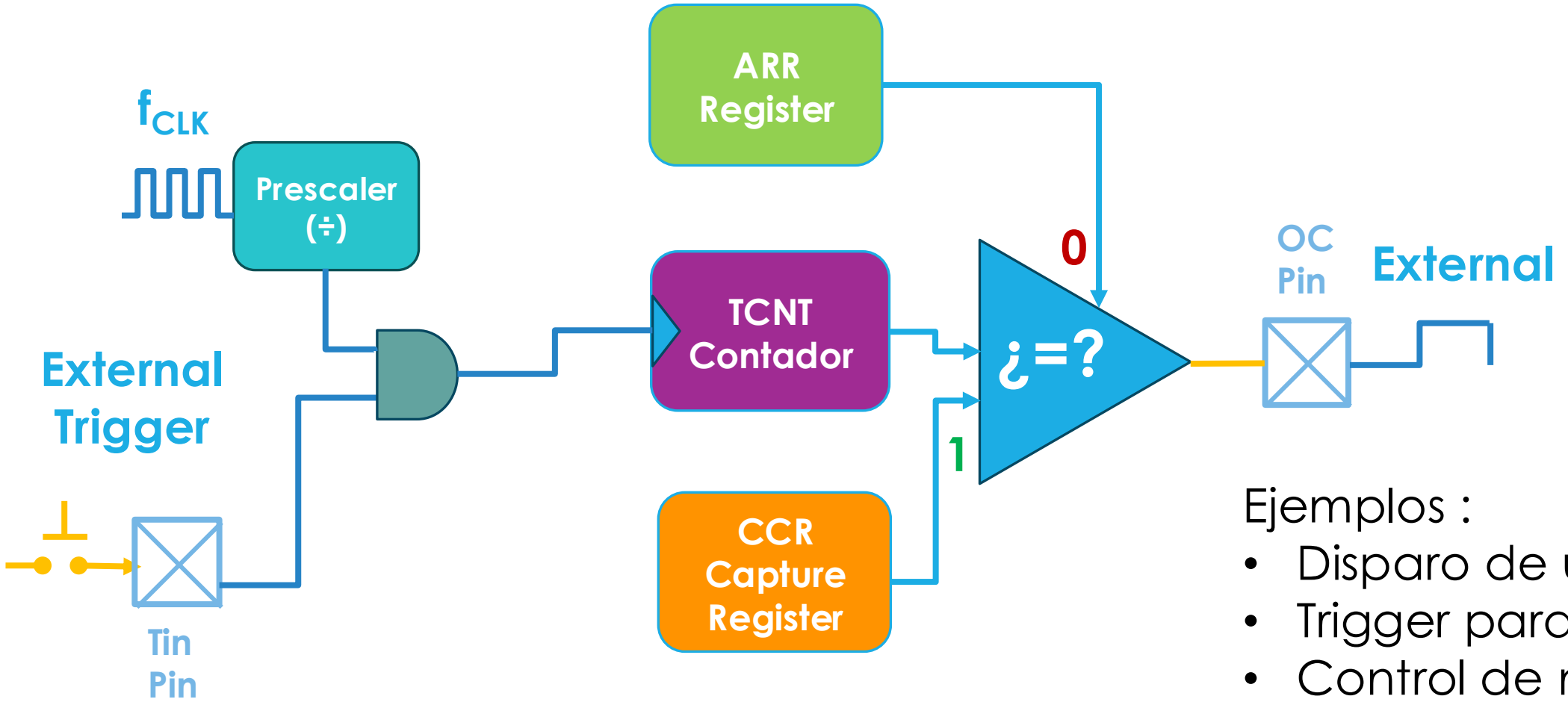




Ejemplos:

- Dimmer LED
- Control de motores
- Salida de audio

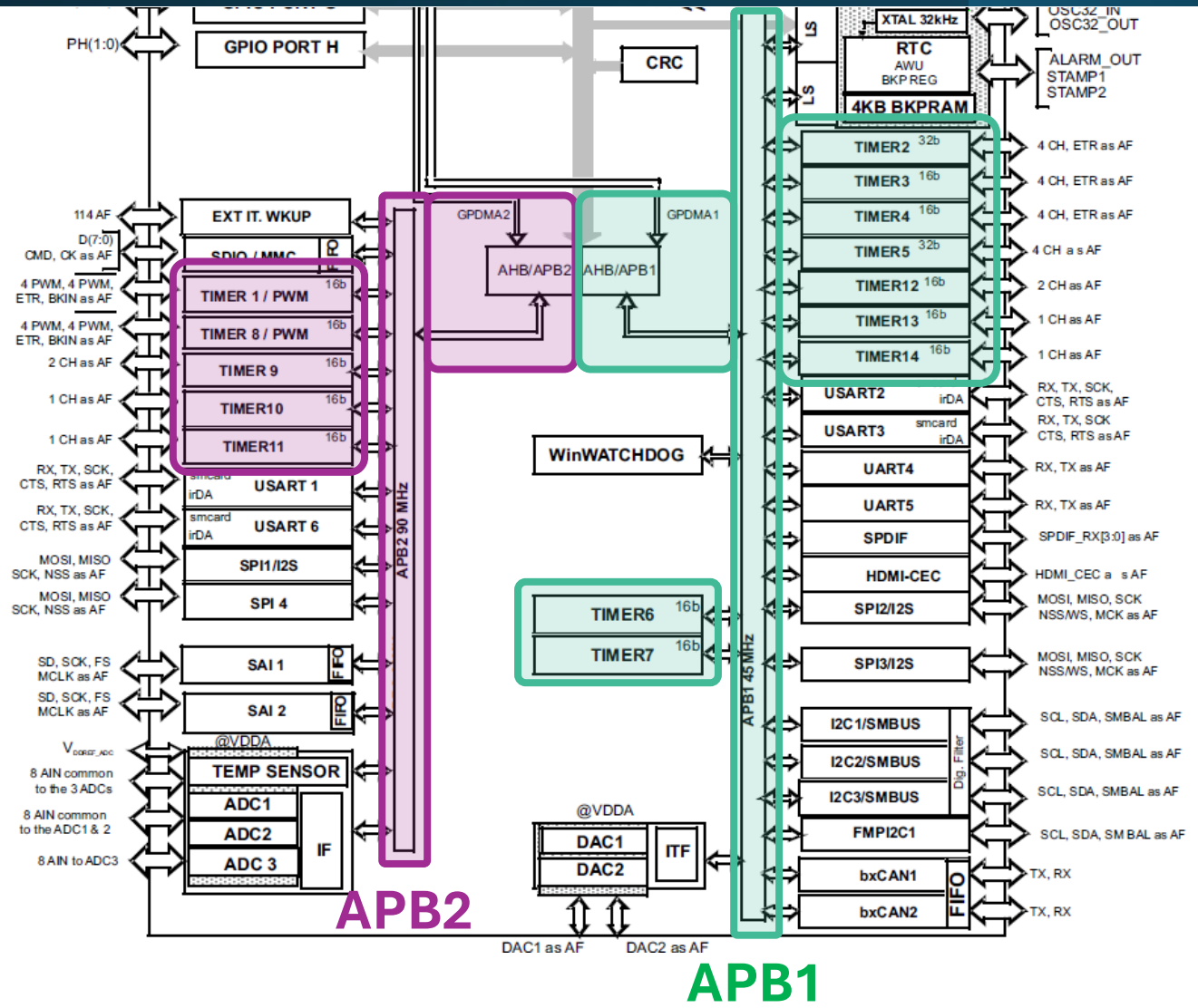
Modo One Pulse



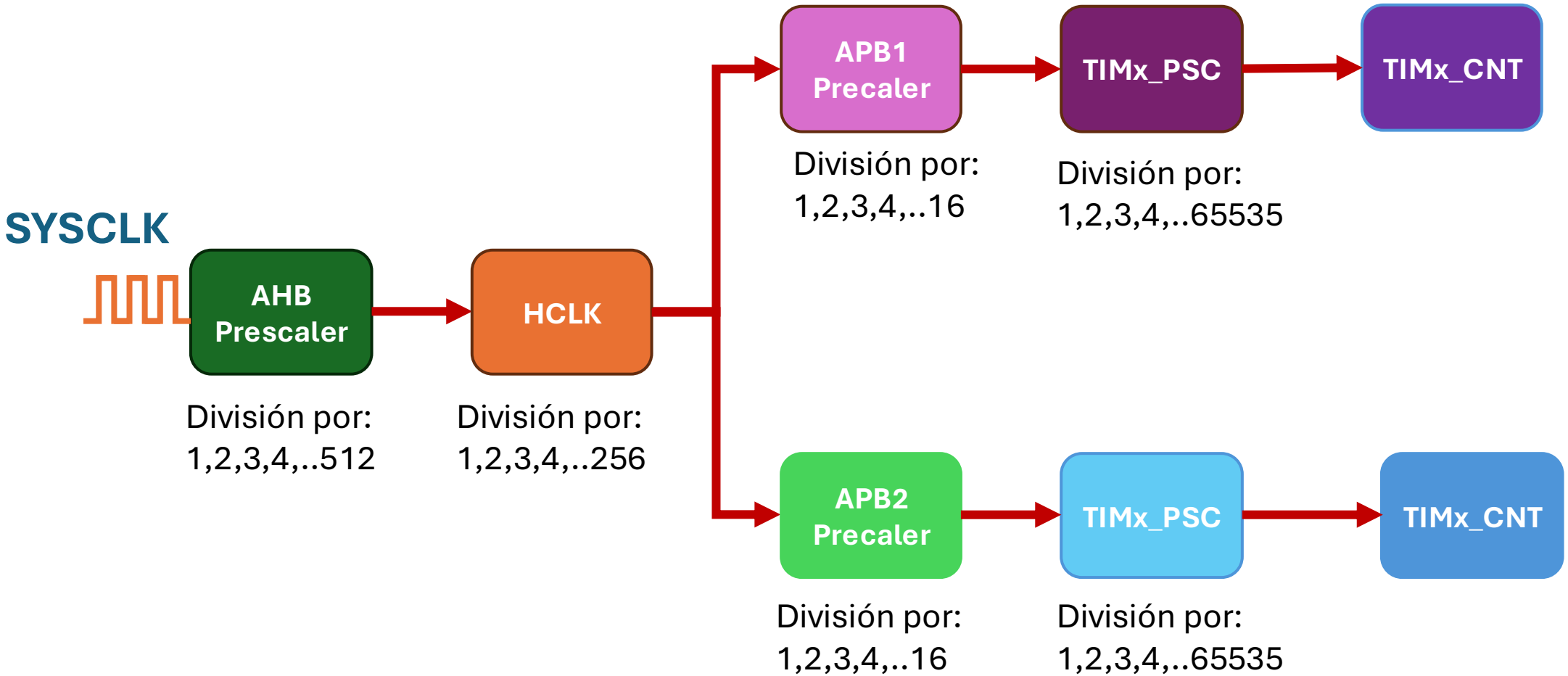
Ejemplos :

- Disparo de ultrasonido
- Trigger para ADC
- Control de motores

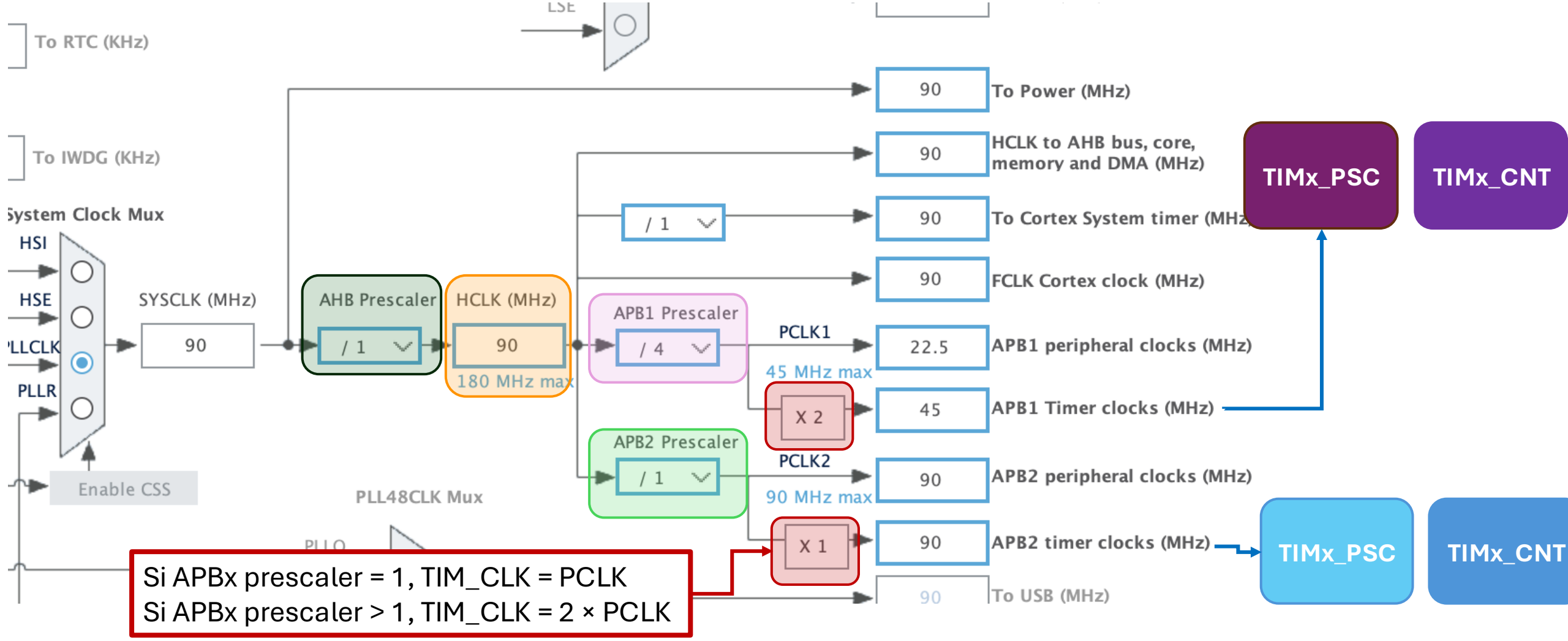
Reloj de Timers



Clock Prescaling



Clock Prescaling



Registros principales de los timers

TIMx_PSC

Registro de prescaler

TIMx_CNT

Registro Contador

TIMx_ARR

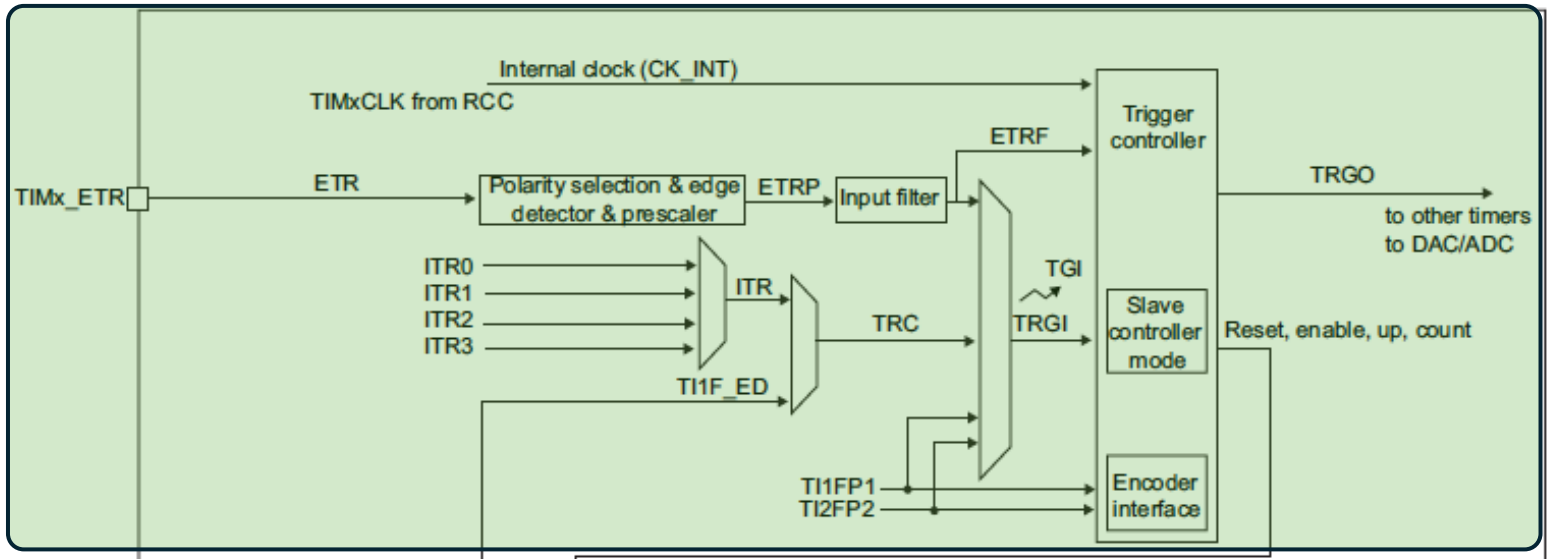
Registro de auto carga

TIMx_CCR

Registro Comparador Capturador

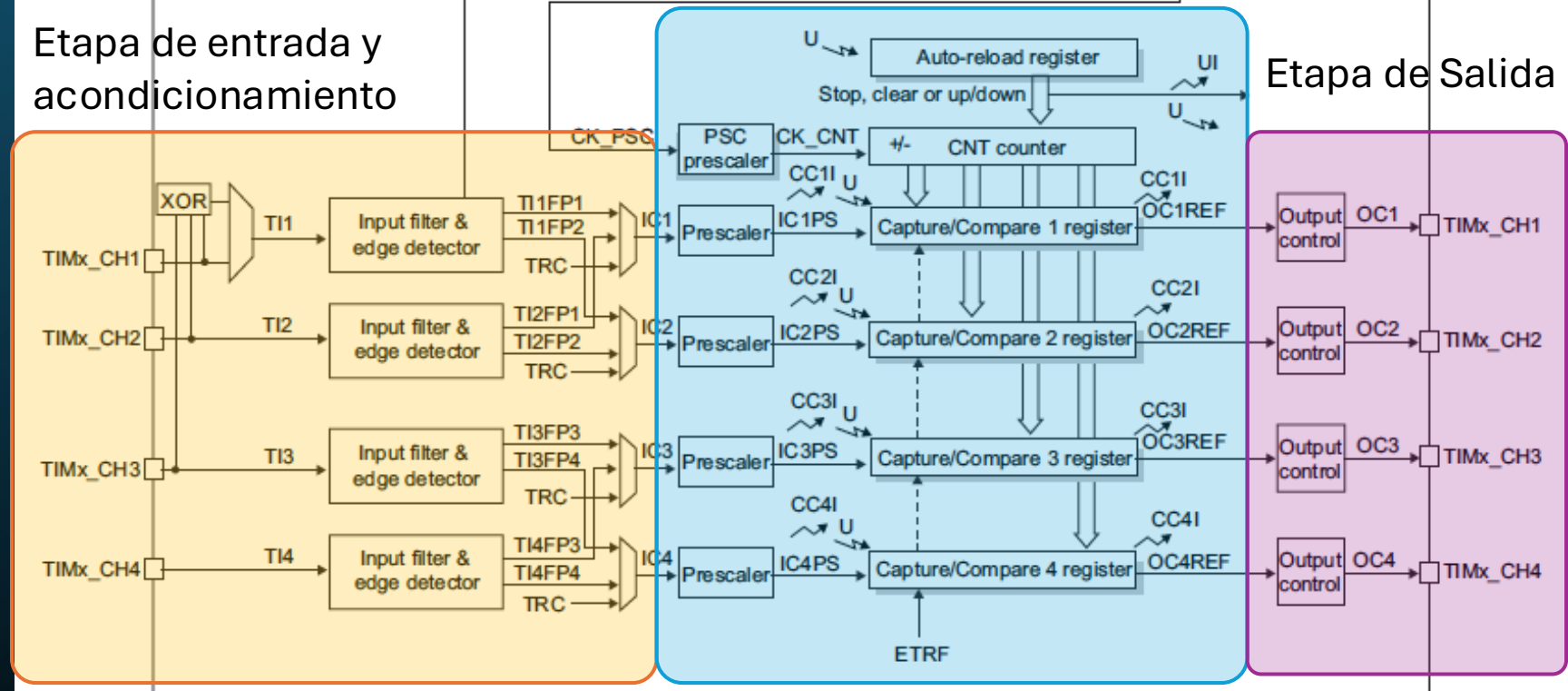
Estructura

Unidad de Sincronización



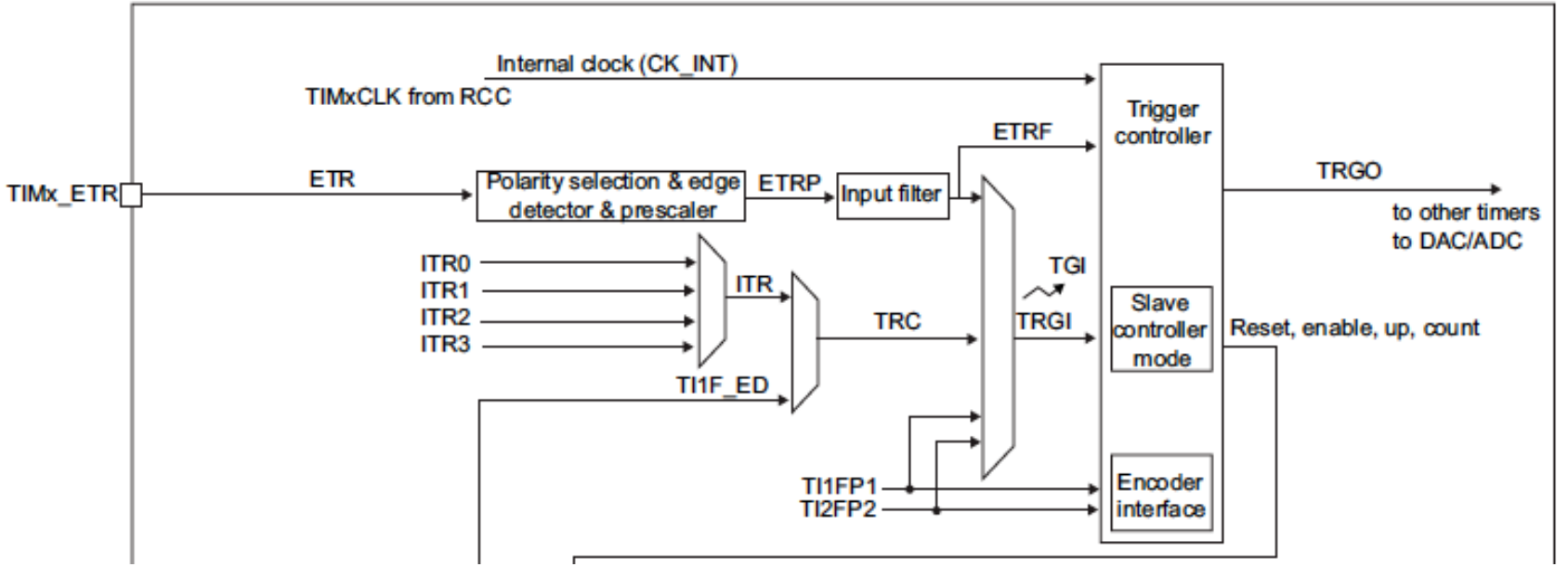
Etapa de entrada y acondicionamiento

Etapa de Salida

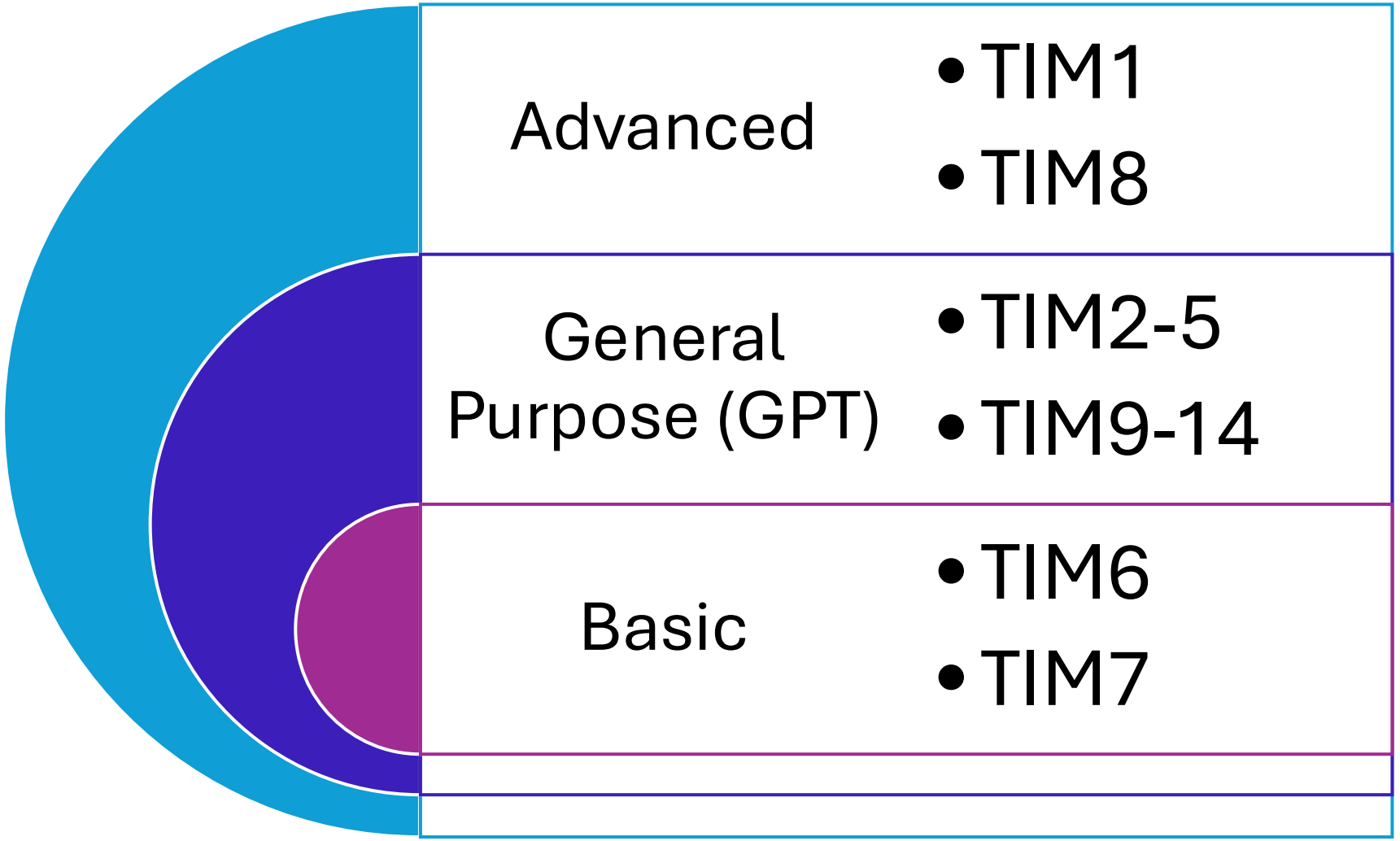


Núcleo del Timer

Fuentes de Reloj



Categorías de Timers



Estructura de configuración de Timers

Configuración

```
typedef struct {
  TIM_TypeDef                 *Instance;     /* Pointer to timer descriptor       */
  TIM_Base_InitTypeDef        Init;          /* TIM Time Base required parameters */
  HAL_TIM_ActiveChannel       Channel;       /* Active channel                    */
  DMA_HandleTypeDef           *hdma[7];      /* DMA Handlers array                */
  HAL_LockTypeDef             Lock;          /* Locking object                    */
  __IO HAL_TIM_StateTypeDef   State;         /* TIM operation state               */
} TIM_HandleTypeDef;
```

Parámetros

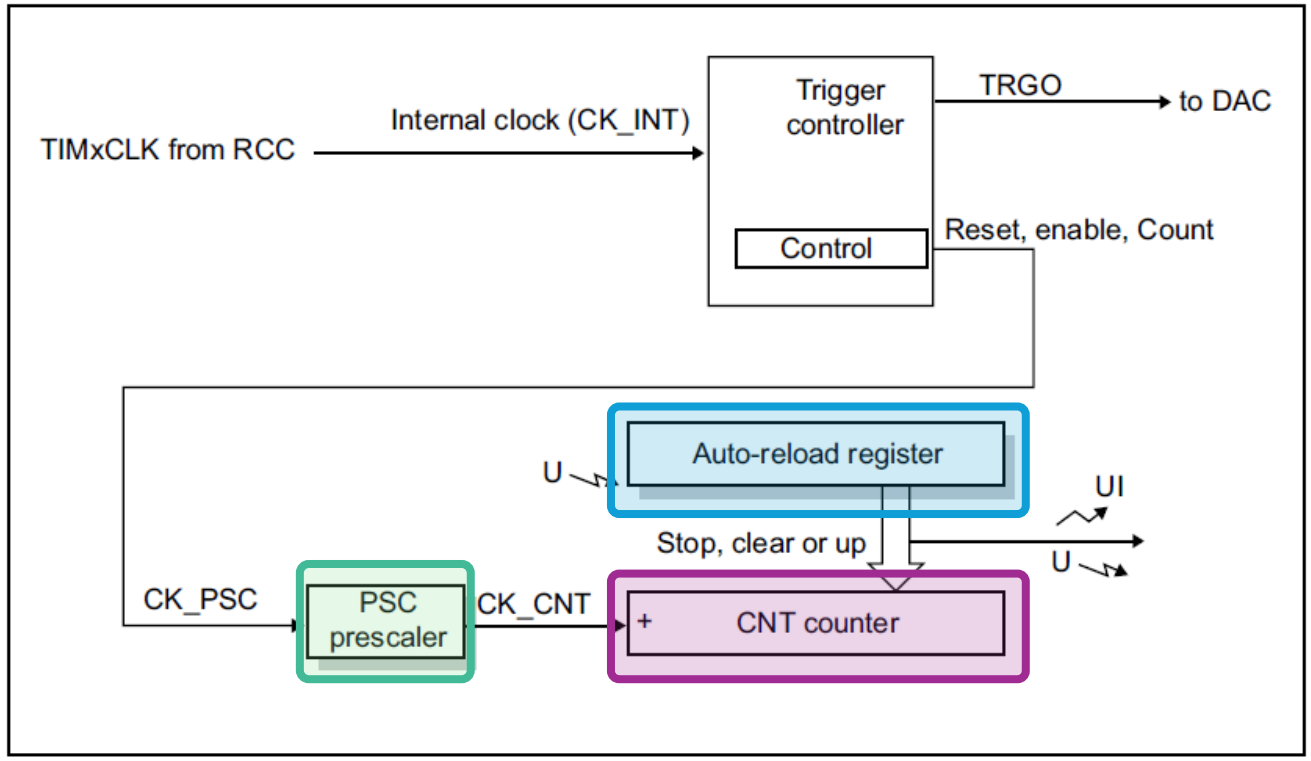
```
typedef struct {
  uint32_t Prescaler;     /* Specifies the prescaler value used to divide the TIM clock. */
  uint32_t CounterMode;   /* Specifies the counter mode.                                 */
  uint32_t Period;        /* Specifies the period value to be loaded into the active
                             Auto-Reload Register at the next update event.              */
  uint32_t ClockDivision; /* Specifies the clock division.                               */
  uint32_t RepetitionCounter; /* Specifies the repetition counter value.                 */
} TIM_Base_InitTypeDef;
```

Timers Básicos

TIM6 y TIM7

Timers Básicos TIM6 y TIM7

- Timers 6 y 7
- Contador para arriba de 16 bits auto-reload
- Prescaler programable de 16 bits
- Circuito de sincronización con el trigger del DAC
- Generación de eventos: ISR/DMA cuando hace overflow



Cálculo de tiempo

$$UpdateEvent = \frac{Timer_{clock}}{(Prescaler + 1)(Period + 1)}$$

$$TIM\_f_{UpdateEvent} = \frac{Timer_{clock}}{(TIM_{Prescaler} + 1)(TIM_{ARR} + 1)}$$

Ejemplo:
Se quiere un tiempo de desborde de
0.5 seg
Reloj APB1 de 48MHz

$$0.5\ seg = \frac{1}{0.5} = 2\ Hz$$

$$T_{OUT} = \frac{(ARR + 1)(PSC + 1)}{F_{CLK}}$$

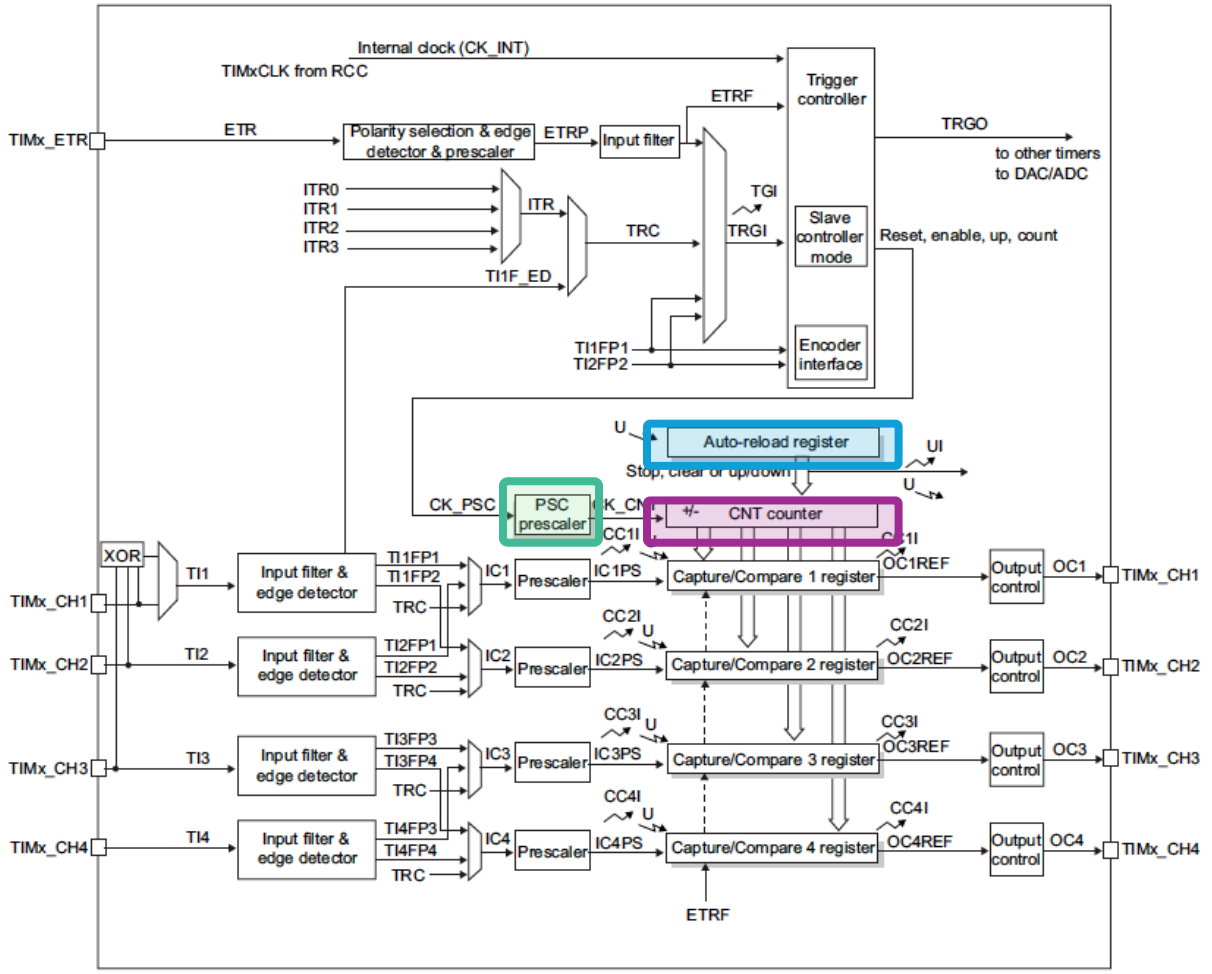
$$2 = \frac{48{,}000{,}000}{(TIM_{Prescaler} + 1)(TIM_{ARR} + 1)}$$

$$TIM_{ARR} = \frac{48{,}000{,}000}{(47{,}999+1)(2)} - 1 = 499$$

Timers de Propósito General GPT

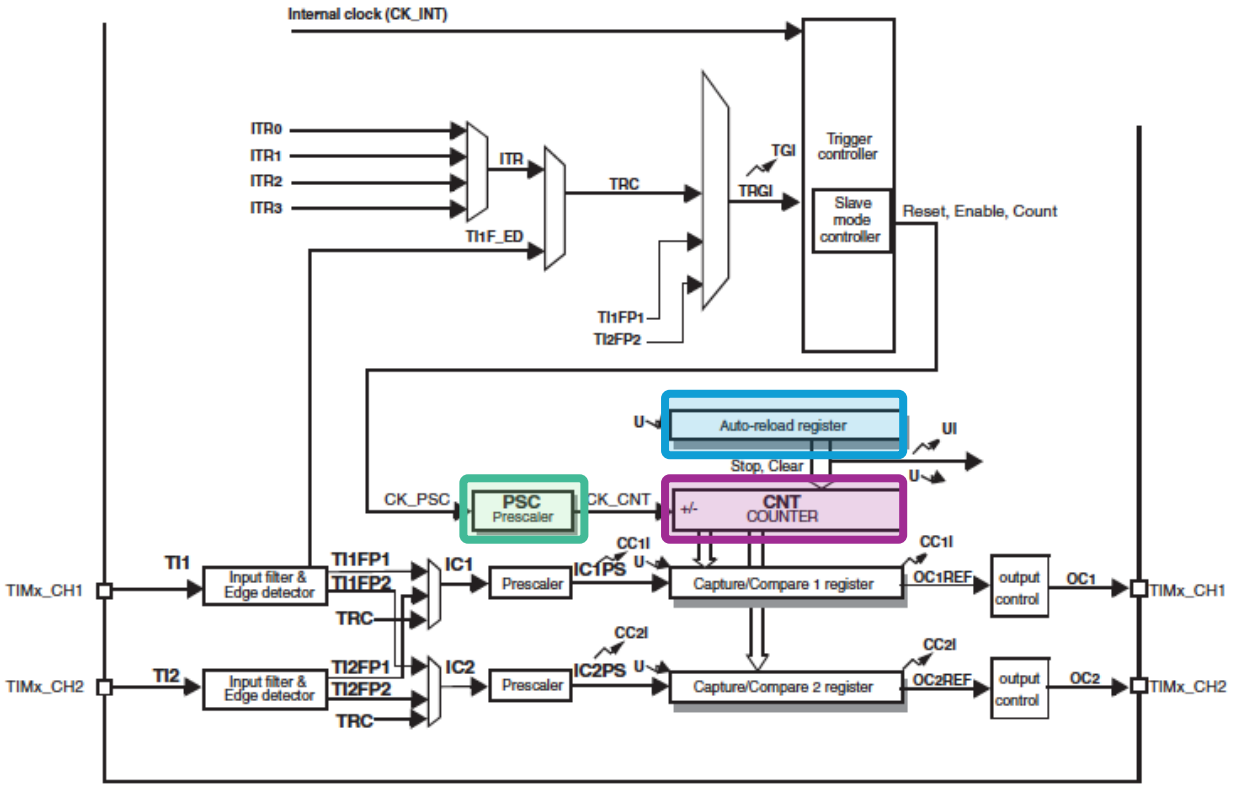
Timers GPT TIM2, TIM3, TIM4 y TIM5

- Timers del 2 al 5
- Contador para arriba/abajo de 16 bits (3 y 4) y 32 bits (2 y 5) auto-reload
- Prescaler programable de 16 bits
- Circuito de sincronización
- Hasta 4 canales independientes que pueden ser utilizados para:
  - Medir señales (input capture)
  - Salida de señales (output compare)
  - Generar señales PWM
  - Un pulso (one-pulse output mode)
- Generación de eventos: Update, Evento Trigger, Input Capture, Output compare

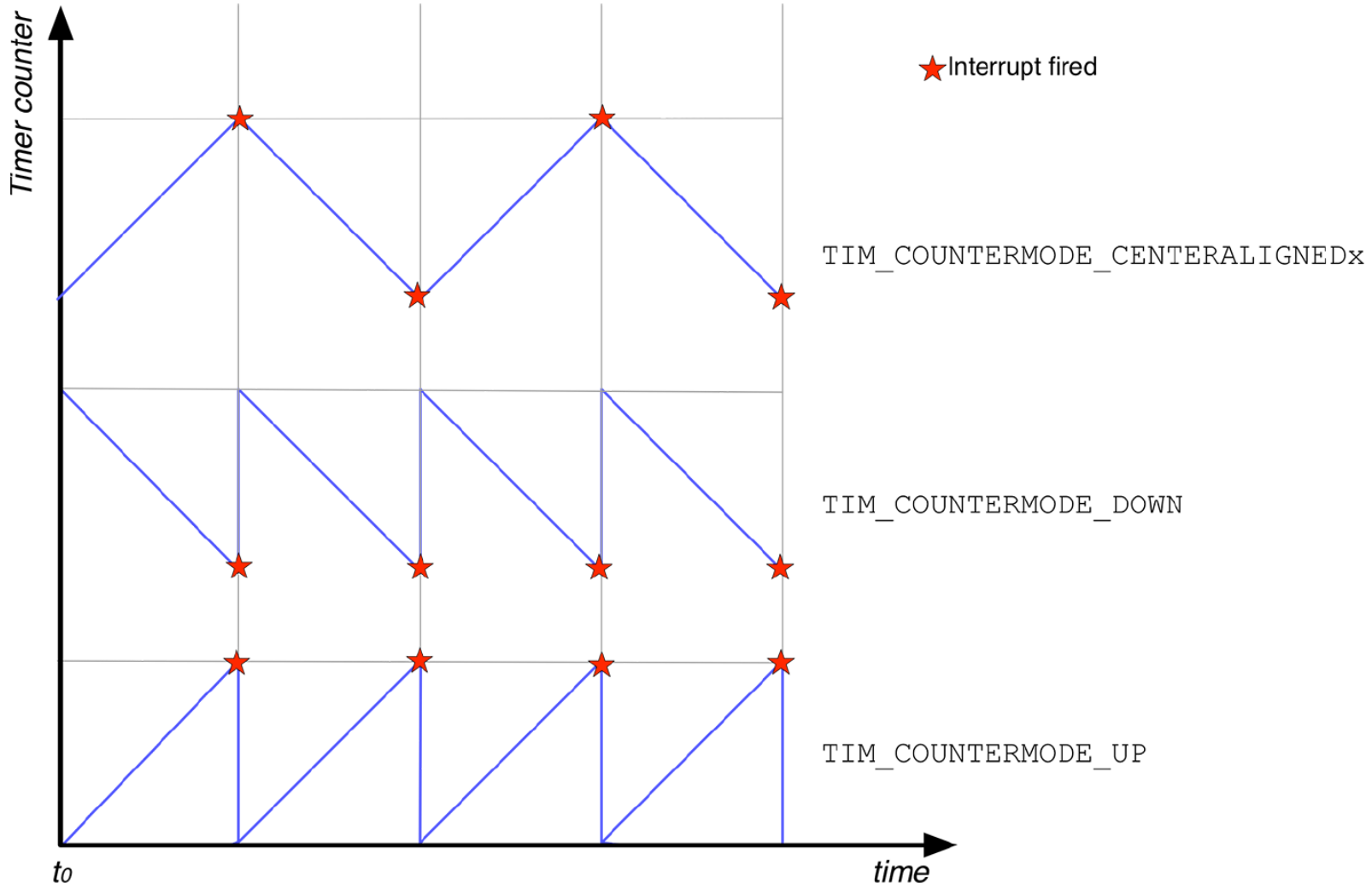


Timers GPT TIM9, TIM10, TIM11, TIM12, TIM13 y TIM14

- Timers del 9 al 14
- Contador para arriba de 16 bits auto-reload
- Prescaler programable de 16 bits
- Circuito de sincronización
- Hasta 2 canales independientes que pueden ser utilizados para:
  - Medir señales (input capture)
  - Salida de señales (output compare)
  - Generar señales PWM
  - Un pulso (one-pulse output mode)
- Generación de eventos: Update, Evento Trigger, Input Capture, Output compare



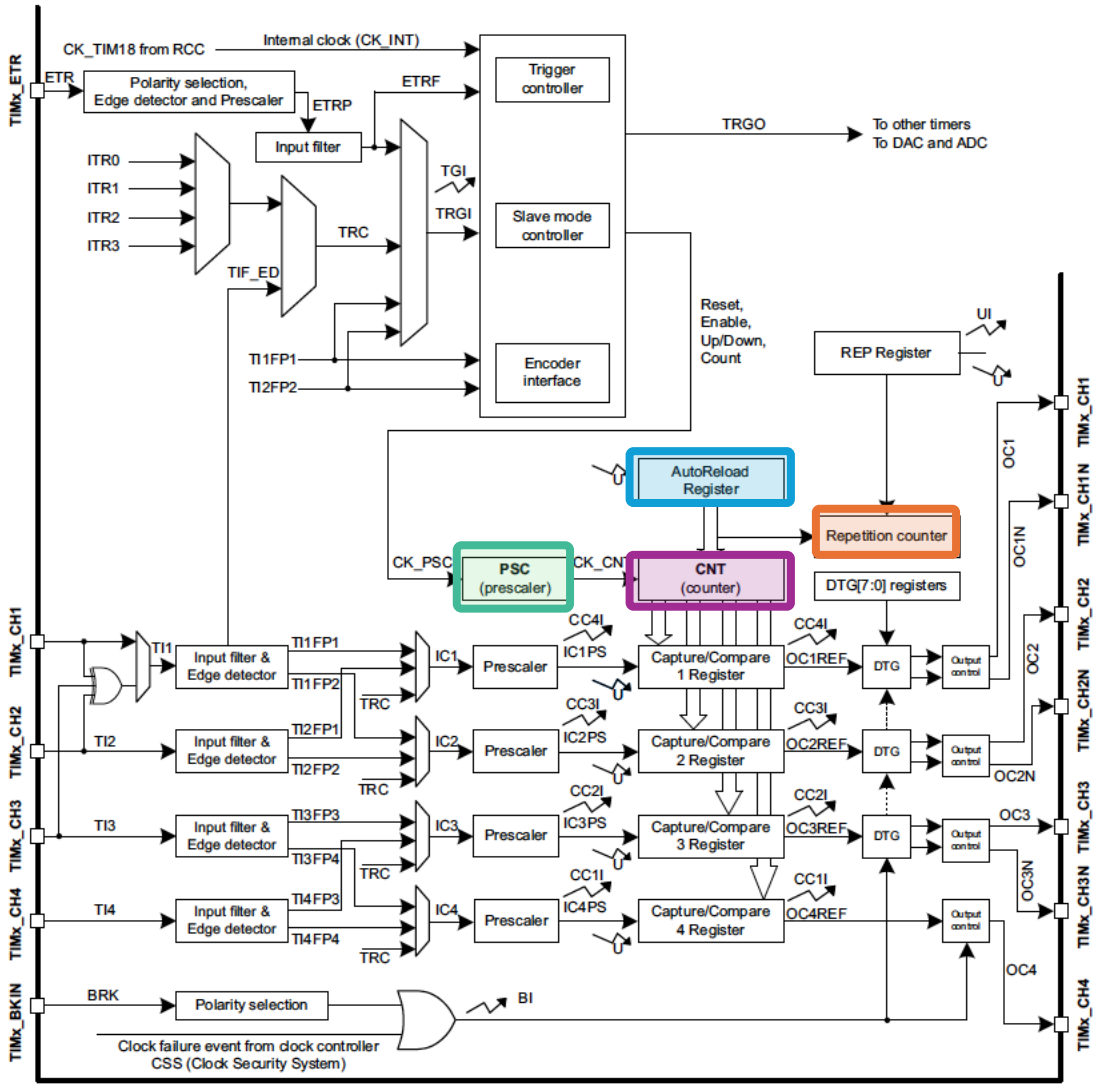
Modos de conteo



Timers de Avanzados

Timers Avanzados TIM1 y TIM8

- Timers 1 y 8
- Contador arriba/abajo de 16 bits
- Prescaler programable de 16 bits
- Circuito de sincronización
- Hasta 4 canales independientes + salidas complementarias que pueden ser utilizados para:
  - Medir señales (input capture)
  - Salida de señales (output compare)
  - Generar señales PWM (Edge and Center-aligned Mode)
  - Un pulso (one-pulse output mode)
- Generación de eventos: Update, Evento Trigger, Input Capture, Output compare, Break input



Cálculo de tiempo

$$UpdateEvent = \frac{Timer_{clock}}{(Prescaler + 1)(Period + 1)(Repetition\ Counter + 1)}$$

$$TIM\_f_{UpdateEvent} = \frac{Timer_{clock}}{(TIM_{Prescaler} + 1)(TIM_{ARR} + 1)(Repetition\ Counter + 1)}$$

Ejemplo:

Se quiere un tiempo de desborde de 0.5 seg

Reloj APB2 de 48MHz

$$T_{OUT} = \frac{(ARR + 1)(PSC + 1)}{F_{CLK}}$$

$$0.5\ seg = \frac{1}{0.5} = 2\ Hz$$

$$2 = \frac{48{,}000{,}000}{(TIM_{Prescaler} + 1)(TIM_{ARR} + 1)(Repetition\ Counter + 1)}$$

Si $Repetition\ Counter = 0$

$$TIM_{ARR} = \frac{48{,}000{,}000}{(47{,}999+1)(0+1)(2)} - 1 = 499$$

Comparación entre timers

Tipo	Canales	PWM	Capture	One Pulse	Break
Básicos	0	No	No	No	No
GPT	4*	Si	Si	Si	No
Avanzados	4 + complementarios	Si	Si	Si	Si

* No todos los GPT tiene 4 canales

Implementación

Implementación Timer Base

Para iniciar el timer:

HAL_TIM_Base_Start();

Para iniciar el timer con ISR:

HAL_TIM_Base_Start_IT();

Para iniciar el timer con DMA:

HAL_TIM_Base_Start_DMA();

Para obtener el valor del timer:

__HAL_TIM_GET_COUNTER(&tim);

Para obtener bandera del timer:

__HAL_TIM_GET_FLAG(&tim);

Para limpiar bandera del timer:

__HAL_TIM_CLEAR_IT(htim, TIM_IT_UPDATE);

Implementación Modo Input Capture

Para iniciar el timer:

*HAL_TIM_IC_Start(TIM_HandleTypeDef *htim, uint32_t Channel);*

Para iniciar el timer con ISR:

*HAL_TIM_IC_Start_IT(TIM_HandleTypeDef *htim, uint32_t Channel);*

Para iniciar el timer con DMA:

*HAL_TIM_IC_Start_DMA(TIM_HandleTypeDef *htim, uint32_t Channel, uint32_t *pData, uint16_t Length);*

Implementación Modo Input Capture

Para obtener el valor del timer:

*HAL_TIM_ReadCapturedValue(const TIM_HandleTypeDef *htim, uint32_t Channel);*

Para parar el timer :

*HAL_TIM_IC_Stop(TIM_HandleTypeDef *htim, uint32_t Channel);*

STM32F4 UART

Comunicación Serial

Debemos de sincronizar para que la información transmitida pueda ser interpretada correctamente.

Comunicación Serial

Para lograr esta comunicación es necesario que ambas dispositivos utilicen la misma frecuencia de reloj (velocidad de transmisión).

El receptor debe conocer del momento en que comienza una nueva palabra.

Esto da lugar a dos modalidades de la comunicación digital:

ASÍNCRONA

SÍNCRONA

Términos importantes

Baudrate: es la velocidad expresada en bits por segundo

Baud rate for standard USART (SPI mode included)

$$\text{Tx/Rx baud} = \frac{f_{CK}}{8 \times (2 - \text{OVER8}) \times \text{USARTDIV}}$$

USARTDIV is an unsigned fixed point number that is coded on the USART_BRR register.

- When OVER8=0, the fractional part is coded on 4 bits and programmed by the DIV_fraction[3:0] bits in the USART_BRR register
- When OVER8=1, the fractional part is coded on 3 bits and programmed by the DIV_fraction[2:0] bits in the USART_BRR register, and bit DIV_fraction[3] must be kept cleared.

Términos importantes

Stop Bit: número de stop bits transmitidos, puede ser 1 o 2

Paridad: indica el modo de paridad, puede ser par o impar, es utilizada para verificar errores del frame.

Parity Mode	Description
UART_PARITY_NONE	No parity check enabled
UART_PARITY_EVEN	The parity bit is set to 1 if the count of bits equal to 1 is odd
UART_PARITY_ODD	The parity bit is set to 1 if the count of bits equal to 1 is even

Modo: especifica si el modo RX o TX está habilitado o

UART Mode	Description
UART_MODE_RX	The UART is configured only in receive mode
UART_MODE_TX	The UART is configured only in transmit mode
UART_MODE_TX_RX	The UART is configured to work bot in receive an transmit mode

Términos importantes

Tamaño de palabra (Word length): especifica el número de bits de datos transmitidos o recibidos, puede ser 8 o 9 bits.

Hardware Flow Control: especifica si el Hardware Flow [illegible] shabilitado.

Flow Control Mode	Description
UART_HWCONTROL_NONE	The Hardware Flow Control is disabled
UART_HWCONTROL_RTS	The *Request To Send*(RTS) line is enabled
UART_HWCONTROL_CTS	The *Clear To Send*(CTS) line is enabled
UART_HWCONTROL_RTS_CTS	Both RTS and CTS lines enabled

UART STM32F4

6 módulos UART

Half / Full Duplex

Control de flujo disponible

Tamaño de palabra de 8 o 9 bits

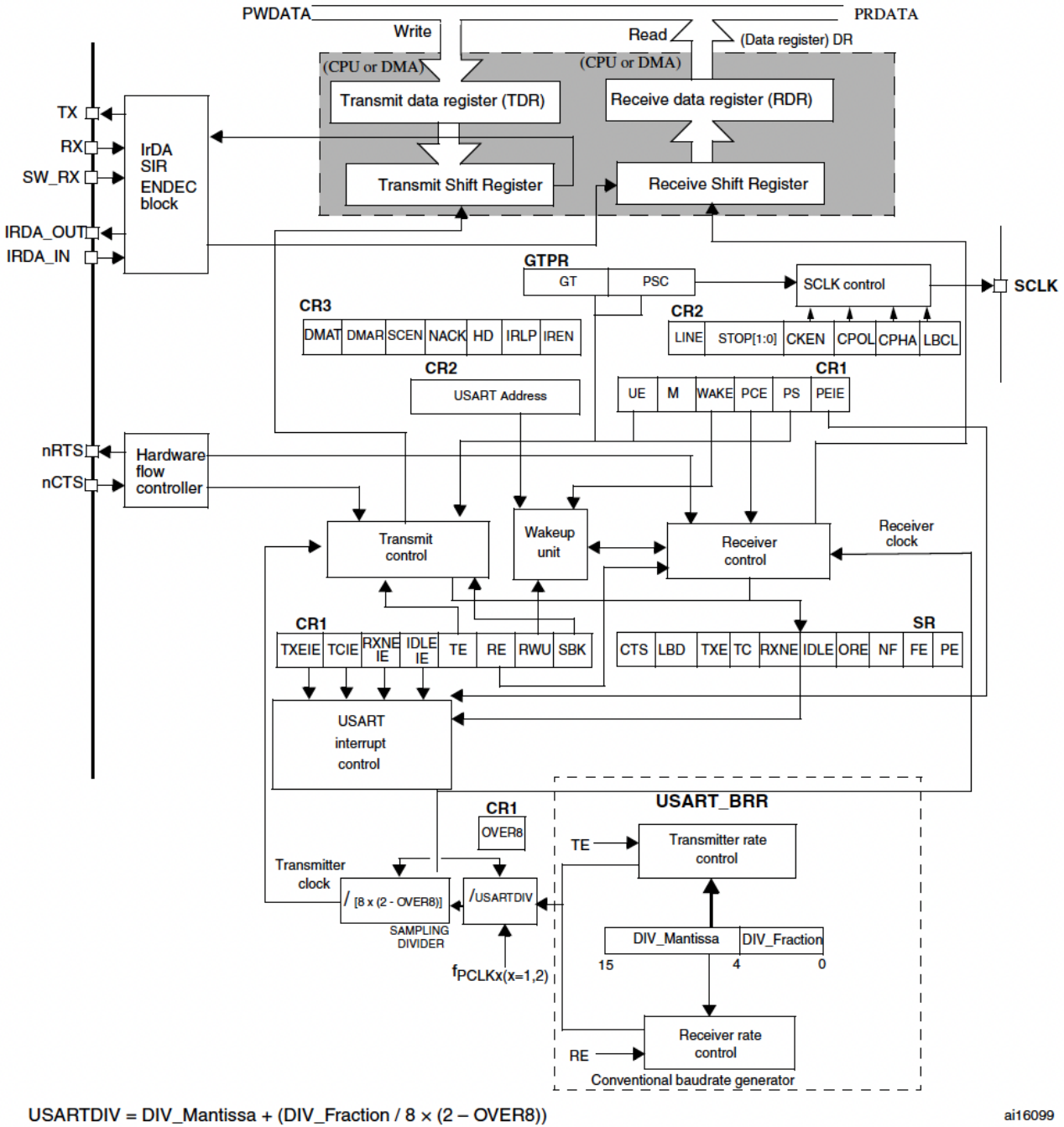
Stop bits configurables 1 o 2 bits

Detection flags

Control de paridad

Detector de 4 errores

10 Fuentes de interrupción



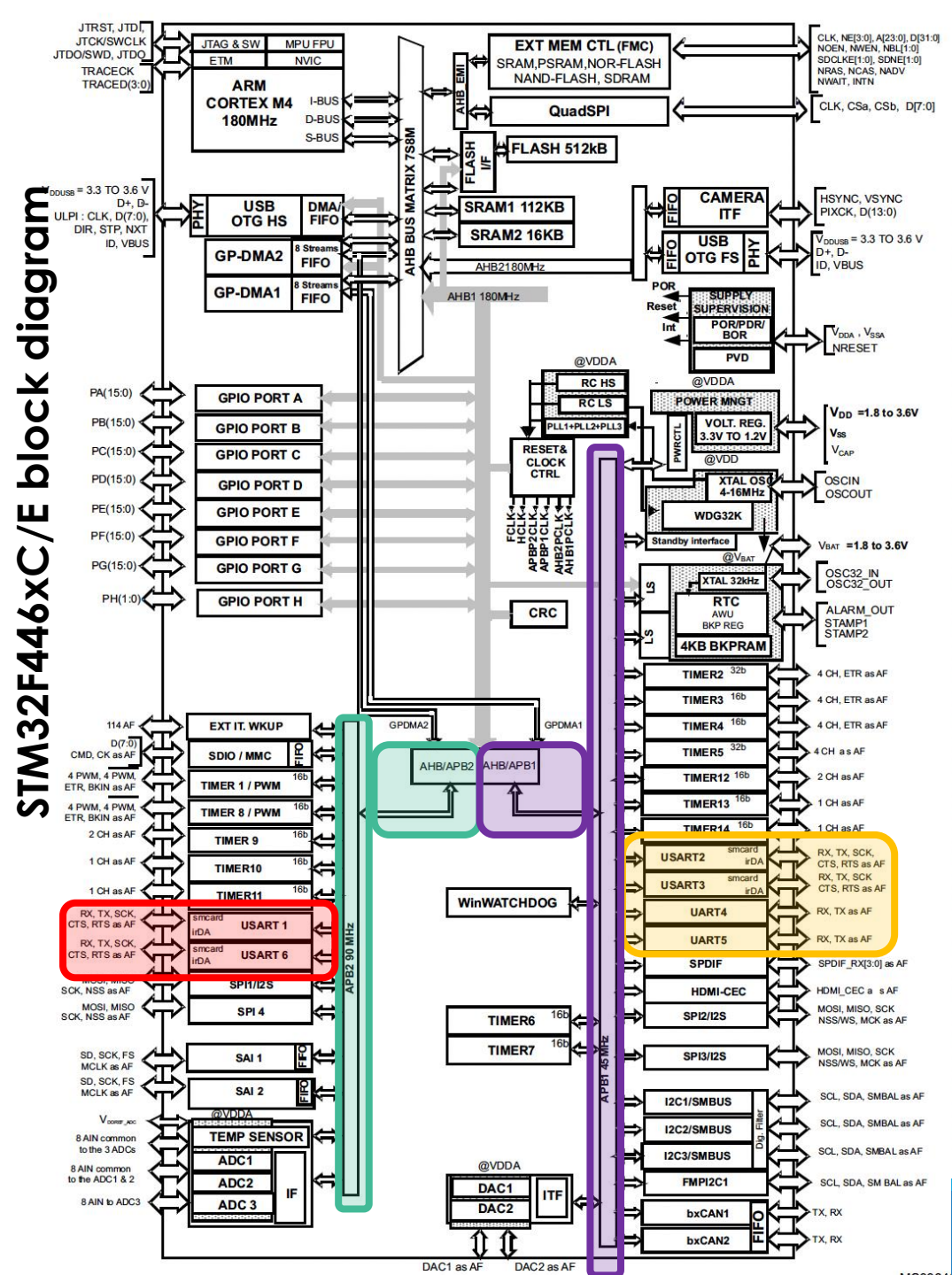
UART STM32F4

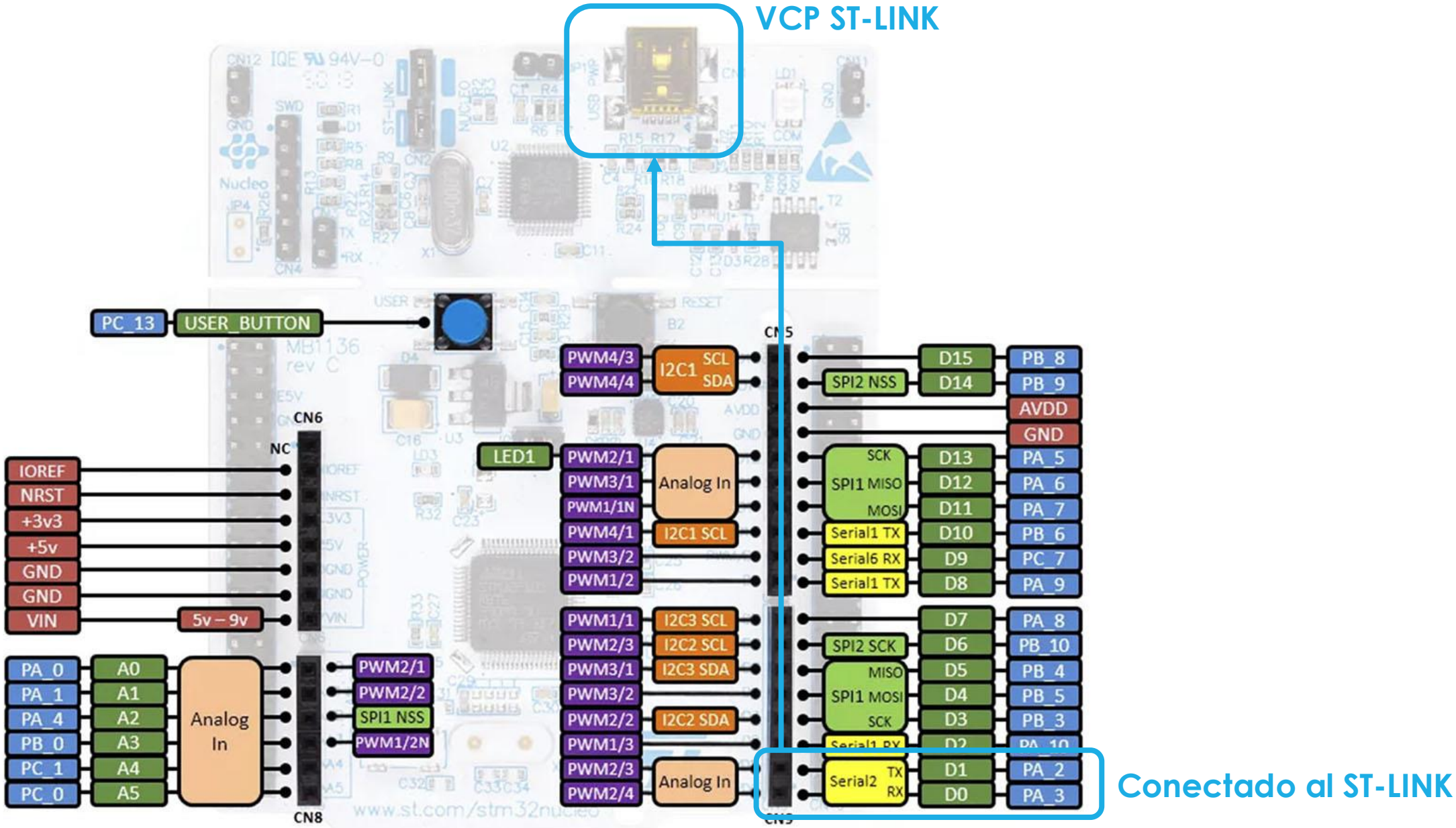
Table 148. USART features

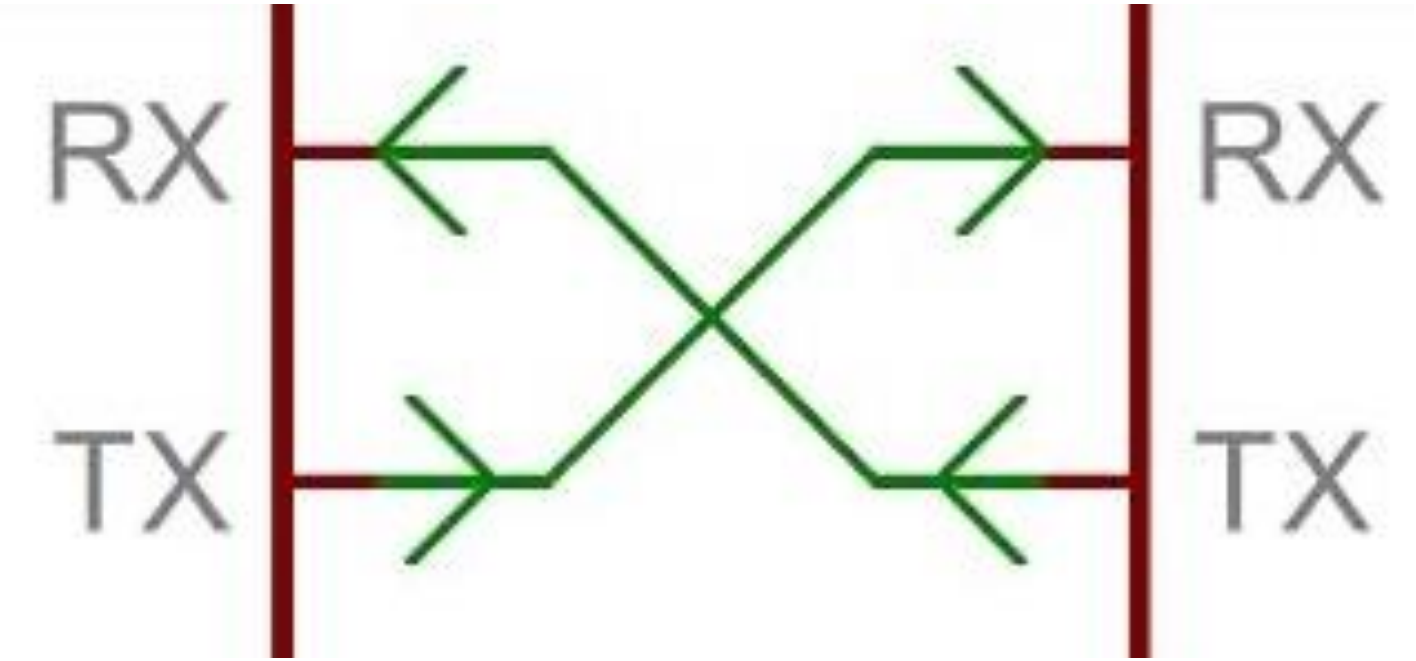
USART modes/features[1]	USART1, USART2, USART3, USART6	UART4, UART5
Hardware flow control for modem	X	X
Continuous communication using DMA	X	X
Multiprocessor communication	X	X
Synchronous mode	X	-
Smartcard mode	X	-
Single-wire half-duplex communication	X	X
IrDA SIR ENDEC block	X	X
LIN mode	X	X
USART data length	8 or 9 bits	

1. X = supported.

STM32F446xC/E block diagram




VCP ST-LINK
PC_13
USER_BUTTON
CN6
NC
IOREF
NRST
+3v3
+5v
GND
GND
VIN
5v – 9v
PA_0
PA_1
PA_4
PB_0
PC_1
PC_0
A0
A1
A2
A3
A4
A5
Analog In
PWM2/1
PWM2/2
SPI1 NSS
PWM1/2N
CN8
LED1
PWM4/3
PWM4/4
I2C1 SCL
SDA
PWM2/1
PWM3/1
PWM1/1N
PWM4/1
PWM3/2
PWM1/2
Analog In
I2C1 SCL
PWM1/1
PWM2/3
PWM3/1
PWM3/2
PWM2/2
PWM1/3
PWM2/3
PWM2/4
I2C3 SCL
I2C2 SCL
I2C3 SDA
I2C2 SDA
Analog In
CN5
SPI2 NSS
SCK
SPI1 MISO
MOSI
Serial1 TX
Serial6 RX
Serial1 TX
SPI2 SCK
MISO
SPI1 MOSI
SCK
Serial2 TX
RX
D15
D14
D13
D12
D11
D10
D9
D8
D7
D6
D5
D4
D3
D2
D1
D0
PB_8
PB_9
AVDD
GND
PA_5
PA_6
PA_7
PB_6
PC_7
PA_9
PA_8
PB_10
PB_4
PB_5
PB_3
PA_2
PA_3
Conectado al ST-LINK



Configuración UART

Estructura de configuración UA

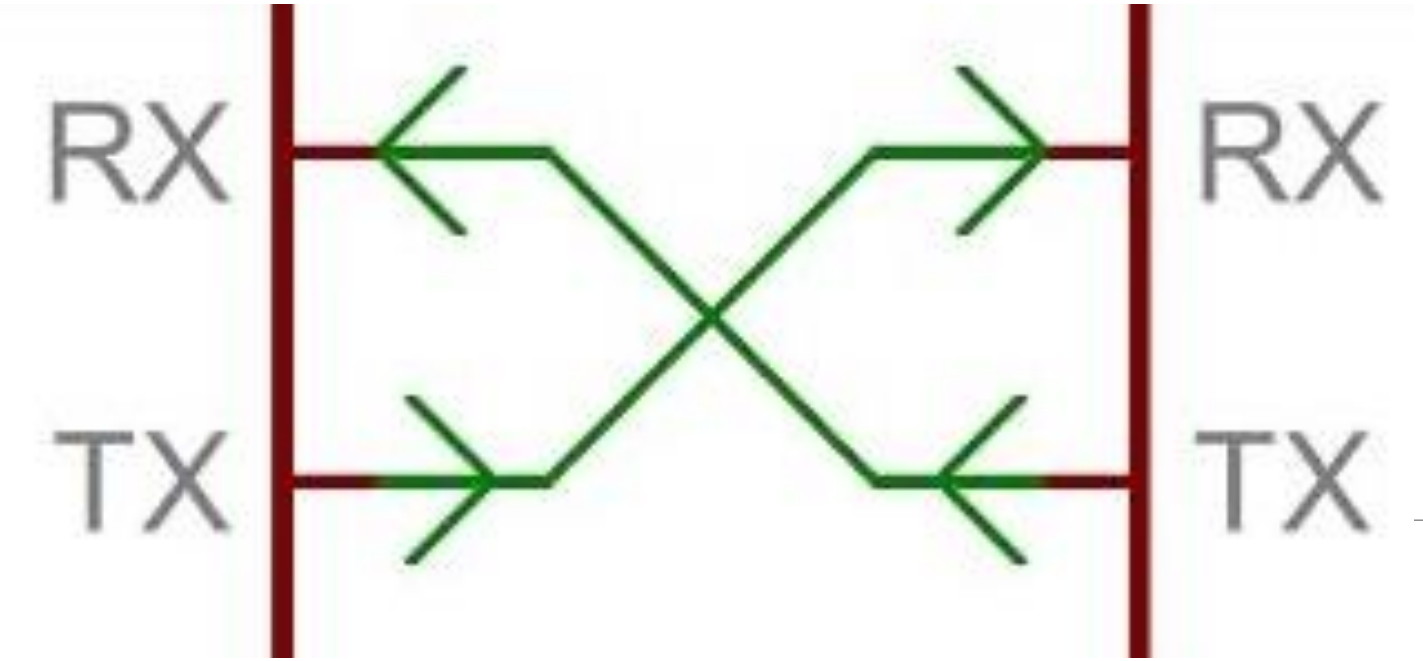

```
typedef struct
{
uint32_t BaudRate; /*!< This member configures the UART communication baud rate.
The baud rate is computed using the following formula:
- IntegerDivider = ((PCLKx) / (8 * (OVR8+1) * (huart->Init.BaudRate)))
- FractionalDivider = ((IntegerDivider - ((uint32_t) IntegerDivider)) * 8 * (OVR8+1)) + 0.5
Where OVR8 is the "oversampling by 8 mode" configuration bit in the CR1 register. */


uint32_t WordLength; /*!< Specifies the number of data bits transmitted or received in a frame.
This parameter can be a value of @ref UART_Word_Length */


uint32_t StopBits; /*!< Specifies the number of stop bits transmitted.
This parameter can be a value of @ref UART_Stop_Bits */


uint32_t Parity; /*!< Specifies the parity mode.
This parameter can be a value of @ref UART_Parity
@note When parity is enabled, the computed parity is inserted
at the MSB position of the transmitted data (9th bit when
the word length is set to 9 data bits; 8th bit when the
word length is set to 8 data bits). */


uint32_t Mode; /*!< Specifies whether the Receive or Transmit mode is enabled or disabled.
This parameter can be a value of @ref UART_Mode */


uint32_t HwFlowCtl; /*!< Specifies whether the hardware flow control mode is enabled or disabled.
This parameter can be a value of @ref UART_Hardware_Flow_Control */


uint32_t OverSampling; /*!< Specifies whether the Over sampling 8 is enabled or disabled, to achieve higher speed (up to fPCLK/8).
This parameter can be a value of @ref UART_Over_Sampling */
} UART_InitTypeDef;
```

Parámetros del módulo UART


```
huart2.Instance = USART2;
huart2.Init.BaudRate = 115200;
huart2.Init.WordLength = UART_WORDLENGTH_8B;
huart2.Init.StopBits = UART_STOPBITS_1;
huart2.Init.Parity = UART_PARITY_NONE;
huart2.Init.Mode = UART_MODE_TX_RX;
huart2.Init.HwFlowCtl = UART_HWCONTROL_NONE;
huart2.Init.OverSampling = UART_OVERSAMPLING_16;
if (HAL_UART_Init(&huart2) != HAL_OK)
{
Error_Handler();
}
```

Configuración de pines de UA


```
GPIO_InitTypeDef GPIO_InitStruct = {0};

if(huart->Instance==USART2)
{
/* USER CODE BEGIN USART2_MspInit 0 */


/* USER CODE END USART2_MspInit 0 */
/* Peripheral clock enable */
__HAL_RCC_USART2_CLK_ENABLE();


__HAL_RCC_GPIOA_CLK_ENABLE();
/**USART2 GPIO Configuration
PA2 ------> USART2_TX
PA3 ------> USART2_RX
*/
GPIO_InitStruct.Pin = GPIO_PIN_2|GPIO_PIN_3;
GPIO_InitStruct.Mode = GPIO_MODE_AF_PP;
GPIO_InitStruct.Pull = GPIO_NOPULL;
GPIO_InitStruct.Speed = GPIO_SPEED_FREQ_VERY_HIGH;
GPIO_InitStruct.Alternate = GPIO_AF7_USART2;
HAL_GPIO_Init(GPIOA, &GPIO_InitStruct);


/* USER CODE BEGIN USART2_MspInit 1 */


/* USER CODE END USART2_MspInit 1 */


}
```



Instrucciones UART

UART STM32F4 HAL Polling

Para transmitir:

HAL_UART_Transmit(UART_HandleTypeDef *huart, uin8t_t *pTxData, uint16_t Size, uint32_t Timeout);

Para recibir:

HAL_UART_Receive(UART_HandleTypeDef *huart, uin8t_t *pRxData, uint16_t Size, uint32_t Timeout);

UART STM32F4 HAL por ISR

Para transmitir:

HAL_UART_Transmit_IT(UART_HandleTypeDef *huart, uin8t_t *pTxData, uint16_t Size);

Para recibir:

HAL_UART_Receive_IT(UART_HandleTypeDef *huart, uin8t_t *pRxData, uint16_t Size);

UART STM32F4 HAL por ISR

Existen 3 fuentes de interrupción disponibles para la transmisión de datos con rutinas predefinidas para cada una de ellas.

Totalidad de datos transmitidos:

HAL_UART_TxCpltCallback(UART_HandleTypeDef *huart)

Mitad de datos transmitidos:

HAL_UART_TxHalfCpltCallback(UART_HandleTypeDef *huart)

Error de periférico

HAL_UART_ErrorCallback(UART_HandleTypeDef *huart)

UART STM32F4 HAL por ISR

Existen 3 fuentes de interrupción disponibles para la recepción de datos con rutinas predefinidas para cada una de ellas.

Totalidad de datos recibidos:

HAL_UART_RxCpltCallback(UART_HandleTypeDef *huart)

Mitad de datos recibidos:

HAL_UART_RxHalfCpltCallback(UART_HandleTypeDef *huart)

Error de periférico

HAL_UART_ErrorCallback(UART_HandleTypeDef *huart)

Bibliografía

Mastering STM32 segunda edición(Carmine Noviello, 2024)

https://www.st.com/en/evaluation-tools/nucleo-f446re.html

https://www.st.com/en/microcontrollers-microprocessors/stm32f446re.html

https://controllerstech.com/stm32-adc-series/

https://controllerstech.com/stm32-uart-series/